

DESAFIO PRÁTICO | DICIONÁRIOS E CONJUNTOS

Chaves, valores, frequências e operações entre conjuntos

Tempo sugerido: 40 minutos | Sem executar código | Justifica as respostas

Nome: _____ Turma: _____ Data: _____

1 CONSULTA DE DICIONÁRIOS

```
notas = {'Ana': 16, 'Rui': 12}
```

Indica o resultado de cada expressão:

```
notas['Ana']
notas.get('Eva', 0)
'Ana' in notas
16 in notas
len(notas)
```

O que acontece ao avaliar notas['Eva']?

2 ATUALIZAR E REMOVER

```
d = {'Ana': 10, 'Rui': 12}
d['Ana'] = 15
d['Eva'] = 18
x = d.pop('Rui')
```

- a) Indica d e x no final.
- b) Qual é a diferença entre a primeira e a segunda atribuição ao dicionário?

3 CONTAR FREQUÊNCIAS

```
texto = 'ana rui ana eva rui ana'
freq = {}
for nome in texto.split():
    freq[nome] = freq.get(nome, 0) + 1
```

- a) Escreve o dicionário final.
- b) Para que serve o valor 0 em get?
- c) Como percorrer os nomes e as contagens em conjunto?

4 OPERAÇÕES ENTRE CONJUNTOS

```
a = {1, 2, 3}
b = {3, 4}
```

- Indica os elementos de cada resultado:
- a | b
- a & b
- a - b
- a ^ b
- A ordem de apresentação dos elementos de um set é garantida?

5 ELEMENTOS ÚNICOS

```
v = [2, 2, 1, 3, 1]
s = set(v)
s.add(3)
s.discard(9)
```

- a) Quais são os elementos finais de s? Qual é len(s)?
- b) Por que add(3) não aumenta o tamanho?
- c) O que mudaria com remove(9) em vez de discard(9)?

6 DESAFIO INTEGRADOR

```
alunos = {101: 'Ana', 102: 'Rui',
          103: 'Eva', 104: 'Leo'}
python = {101, 102, 103}
bd = {102, 104}
```

- a) Mostra número e nome dos inscritos em ambas as disciplinas, por número.
- b) Indica os inscritos só em Python e o total de alunos em pelo menos uma disciplina.

DESAFIO PRÁTICO | DICIONÁRIOS E CONJUNTOS

Chaves, valores, frequências e operações entre conjuntos

SOLUÇÕES

Compara os resultados e verifica o raciocínio.

1 CONSULTA DE DICIONÁRIOS

Resultados: **16, 0, True, False, 2.**

O operador in testa as **chaves**. O número 16 é um valor, não uma chave deste dicionário.

notas['Eva'] provoca **KeyError**, porque a chave não existe. get permite fornecer um valor predefinido, sem inserir a chave.

2 ATUALIZAR E REMOVER

```
d = {'Ana': 15, 'Eva': 18}
x = 12
```

d['Ana'] = 15 **atualiza** o valor de uma chave existente.

d['Eva'] = 18 **adiciona** um novo par chave-valor.

pop remove o par associado a Rui e devolve o seu valor. Uma chave identifica um único valor no dicionário.

3 CONTAR FREQUÊNCIAS

```
{'ana': 3, 'rui': 2, 'eva': 1}
```

b) Se o nome ainda não existe, get devolve 0. Somar 1 inicia a contagem. Nas ocorrências seguintes, incrementa o valor existente.

```
for nome, quantidade in freq.items():
    print(nome, quantidade)
```

c) items() permite percorrer cada par chave-valor.

4 OPERAÇÕES ENTRE CONJUNTOS

União: {1, 2, 3, 4}.

Interseção: {3}.

Diferença a - b: {1, 2}.

Diferença simétrica: {1, 2, 4}.

A ordem de apresentação de um set **não é garantida**. Os resultados acima descrevem os elementos, não uma ordem obrigatória.

Para obter uma lista ordenada, pode usar-se sorted(conjunto).

5 ELEMENTOS ÚNICOS

a) Os elementos são {1, 2, 3}; len(s) é 3.

b) Um conjunto não conserva duplicados. O elemento 3 já existia.

c) remove(9) provocaria **KeyError**, porque 9 não pertence ao conjunto. discard(9) não provoca erro quando o elemento está ausente.

6 DESAFIO INTEGRADOR

```
comuns = python & bd
for numero in sorted(comuns):
    print(numero, alunos[numero])
```

a) Saída: **102 Rui**.

b) Só em Python: python - bd = {101, 103}, correspondentes a Ana e Eva.

Em pelo menos uma disciplina: python | bd = {101, 102, 103, 104}. Total: **4 alunos**.

O dicionário associa números a nomes; os conjuntos representam inscrições sem duplicados.